

Informe Técnico

Ruta óptima entre ciudades mediante grafos ponderados

Proyecto · Matemática Discreta

Integrantes:

- Lucas Villegas
- Piero Soto
- Nicolás Calderón
- Mauricio Valdés

Introducción

El presente informe técnico detalla el diseño y desarrollo de una solución computacional basada en la teoría de grafos para resolver el problema del camino mínimo. El problema abordado consiste en modelar una red simplificada de ciudades alemanas conectadas por carretera y encontrar la ruta óptima entre una ciudad de origen y una ciudad de destino.

Para este proyecto se seleccionaron 15 ciudades de Alemania, conectadas mediante 25 rutas reales por carretera. El criterio de ponderación utilizado fue la distancia aproximada en kilómetros, consultada en OpenStreetMap Directions utilizando el motor OSRM. La solución se implementó en Python mediante una interfaz gráfica que permite seleccionar nodos, calcular la ruta mínima y visualizar el camino encontrado.

Repositorio público del proyecto: <https://github.com/PieroSotoUCT/rutas-grafos>

Modelamiento matemático

La red de ciudades se modeló mediante un grafo ponderado y no dirigido. Formalmente, el grafo se define como:

$$G = (V, E, w)$$

Donde cada componente representa un elemento de la red:

- Conjunto de vértices (V): corresponde a las 15 ciudades seleccionadas: Hamburg, Bremen, Hannover, Berlin, Leipzig, Dresden, Dortmund, Essen, Dusseldorf, Cologne, Frankfurt, Mannheim, Stuttgart, Nuremberg y Munich. Por lo tanto, $|V| = 15$.
- Conjunto de aristas (E): corresponde a las 25 conexiones por carretera entre pares de ciudades. Como las rutas se consideran transitables en ambos sentidos, el grafo es no dirigido. Por lo tanto, si (u, v) pertenece a E , entonces (v, u) también se considera parte de la conexión.
- Función de peso (w): asigna a cada arista un número real positivo. En este proyecto, $w(u, v)$ representa la distancia aproximada en kilómetros entre dos ciudades conectadas. Se expresa como $w: E \rightarrow \mathbb{R}^+$.

Propiedad de conectividad

Para que exista una solución entre cualquier ciudad de origen y destino, el grafo debe ser conexo. En este caso, el grafo construido permite llegar desde cualquier ciudad seleccionada hacia otra mediante una secuencia de conexiones. Esta propiedad garantiza que el algoritmo pueda calcular una ruta válida dentro de la red.

Caminos y ruta óptima

Dadas una ciudad de origen s y una ciudad de destino t , un camino P se define como una secuencia de ciudades conectadas: $P = (s = v_0, v_1, v_2, \dots, v_k = t)$. El costo total del camino corresponde a la suma de las distancias de sus aristas: $C(P) = w(v_0, v_1) + w(v_1, v_2) + \dots + w(v_{k-1}, v_k)$.

La ruta óptima P^* corresponde al camino que minimiza la distancia total entre todas las rutas posibles que conectan el origen con el destino.

Algoritmo de camino mínimo

Para resolver el problema se seleccionó el algoritmo de Dijkstra. Este algoritmo permite encontrar caminos mínimos en grafos ponderados con pesos no negativos, revisando y actualizando las distancias acumuladas desde un nodo de origen hasta los demás nodos o hasta un destino específico (DataCamp, 2024).

Funcionamiento y pasos principales

- Inicialización: se asigna distancia 0 a la ciudad de origen y distancia infinita a las demás ciudades.
- Selección: se elige la ciudad no visitada con la menor distancia acumulada conocida.
- Relajación de aristas: se revisan los vecinos de la ciudad actual y se actualiza la distancia si se encuentra un camino más corto.
- Reconstrucción: al llegar al destino, se usa el registro de ciudades anteriores para reconstruir la ruta mínima.

Justificación del algoritmo

Dijkstra fue elegido porque todas las distancias por carretera son positivas. Esto cumple con la condición principal del algoritmo: trabajar con pesos no negativos. A diferencia de Bellman-Ford, que es útil para grafos con pesos negativos, en este proyecto no se requiere procesar ese tipo de valores.

La implementación realizada no utiliza colas de prioridad ni librerías externas de grafos. El algoritmo fue construido manualmente mediante diccionarios, ciclos, conjuntos y listas de adyacencia. Debido al tamaño reducido del grafo, compuesto por 15 ciudades y 25 conexiones, esta implementación resulta suficiente para calcular las rutas de manera clara y eficiente.

Implementación

La aplicación fue desarrollada en Python, separando la lógica del algoritmo de la interfaz visual. El módulo lógico contiene la carga de datos desde el archivo CSV, la construcción del grafo y la función `dijkstra(grafo, origen, destino)`. El módulo visual utiliza `tkinter` y `tkk` para construir la interfaz gráfica, además de `Canvas` para dibujar nodos, aristas y resaltar la ruta encontrada.

El grafo se representó mediante un diccionario de listas de adyacencia. Cada clave del diccionario corresponde a una ciudad y su valor asociado es una lista de tuplas con sus vecinos y las distancias respectivas. Por ejemplo: `{"Berlin": [("Hannover", 290), ("Leipzig", 190)]}`.

Fragmento de código relevante

El siguiente fragmento muestra la parte central de la función `Dijkstra`, donde se inicializan las estructuras y se realiza la relajación de aristas:

```
def dijkstra(grafo, origen, destino):
    distancias = {}
    anteriores = {}
    visitados = set()

    for ciudad in grafo:
        distancias[ciudad] = float("inf")
        anteriores[ciudad] = None

    distancias[origen] = 0

    while len(visitados) < len(grafo):
        ciudad_actual = buscar_ciudad_menor_distancia(distancias, visitados)

        if ciudad_actual is None:
            break

        if ciudad_actual == destino:
            break

        visitados.add(ciudad_actual)

        for vecino, peso in grafo[ciudad_actual]:
            nueva_distancia = distancias[ciudad_actual] + peso

            if nueva_distancia < distancias[vecino]:
                distancias[vecino] = nueva_distancia
                anteriores[vecino] = ciudad_actual

    ruta = reconstruir_ruta(anteriores, origen, destino)
    return ruta, distancias[destino]
```

Este fragmento permite identificar los elementos principales del algoritmo: distancias guarda el menor costo conocido, anteriores permite reconstruir la ruta y visitados evita repetir ciudades ya procesadas.

Resultados y pruebas

Para verificar el funcionamiento de la aplicación, se realizaron pruebas con distintas ciudades de origen y destino. Los resultados muestran la ruta calculada y su distancia total:

Origen -> destino	Ruta óptima	Distancia total
Berlin -> Munich	Berlin -> Leipzig -> Nuremberg -> Munich	642 km
Hamburg -> Cologne	Hamburg -> Hannover -> Dortmund -> Cologne	457 km
Dresden -> Essen	Dresden -> Leipzig -> Hannover -> Dortmund -> Essen	629 km

Conclusión

El desarrollo de este proyecto permitió aplicar conceptos de Matemática Discreta a un problema computacional concreto. La red de ciudades se representó mediante un grafo ponderado y no dirigido, lo que permitió analizar rutas, pesos, conectividad y caminos mínimos de forma aplicada.

Además, la implementación manual de Dijkstra permitió comprender cómo las estructuras de datos influyen en el funcionamiento del algoritmo. La aplicación final permite seleccionar ciudades, calcular la ruta mínima y visualizar el camino resultante en el grafo.

Como mejoras futuras, se podría incorporar un mapa geográfico real, permitir la carga dinámica de nuevas ciudades o modificar la función de peso para minimizar otros criterios, como tiempo de viaje o costo monetario.

Nota de apoyo

ChatGPT se utilizó como herramienta de asistencia para revisión ortográfica, redacción y organización del informe. El contenido técnico, el código, los datos del grafo y la implementación fueron desarrollados y revisados por el equipo.

Referencias

DataCamp. (2024). *Implementación del algoritmo de Dijkstra en Python: Un tutorial paso a paso.*

<https://www.datacamp.com/es/tutorial/dijkstra-algorithm-in-python>

OpenStreetMap contributors. (s. f.). *OpenStreetMap Directions.*

<https://www.openstreetmap.org/directions>

Project OSRM. (s. f.). *Open Source Routing Machine.*

<https://project-osrm.org/>

OpenAI. (2026). *ChatGPT [Herramienta de asistencia para revisión ortográfica y redacción].*

<https://chatgpt.com/>